

Project Plan
*Benchmarking and Securing Language Models Against
Adversarial Prompt Attacks*

Team XC2/WP1

Name	Position	email	phone
Vaisakh Pariyacheri	Team Lead	104800822@student.swin.edu.au	0432111564
Aarathi Narayanan Kadambatta	Developer	104801951@student.swin.edu.au	0421566220
Aayudh Himanshubhai Vaghani	Developer	104996006@student.swin.edu.au	0403882904
Arun Ragavendhar Arunachalam Palaniyappan	Tech Lead, Developer	104837257@student.swin.edu.au	0410692290
Samsun Gulshan Sheik Dawood	Research Lead, Developer	105009251@student.swin.edu.au	0449941039
Vikas Kadruvan	Documentation Lead	104981442@student.swin.edu.au	0423160340

COS80029
Technology Application Project
2nd Semester
5 September 2025

Contents

Document Change Control	3
Document Sign Off.....	4
Client Sign off.....	4
1. Introduction	4
1.1. Background.....	5
1.2. Key Project Personnel.....	5
2. Terms of Reference.....	6
2.1. Objectives.....	6
2.2. Scope	7
2.3. Critical Success Factors	7
2.4. Acceptance Criteria	8
3. Establishment.....	8
3.1. Processes, Procedures and Standards.....	9
3.2. Project Environment.....	12
3.3. Project Team Skill Development Requirements.....	12
4. Deliverables, Activities and Capital Resources.....	12
4.1. Deliverables.....	12
4.2. Activities.....	13
4.3. Resources	14
5. Organisation and Structure.....	14
6. Risks.....	16
7. Schedule	18
7.1. Project Timeline.....	18
7.2. External Dependencies	19
7.3. Assumptions	19
8. Budget	19
9. References	20

DOCUMENT CHANGE CONTROL

Version	Date	Authors	Summary of Changes
1	04/09/2025	All team members	Initial draft
2	05/09/2025	All team members	Finalized the document.
3	06/09/2025	All team members	Formatted the document, labelled figures and tables

DOCUMENT SIGN OFF

Name	Position	Signature	Date
Vaisakh Pariyacheri	Team Lead	VP	06/09/25
Aarathi Narayanan Kadambatta	Developer	ANK	06/09/25
Aayudh Himanshubhai Vaghani	Developer	AHV	06/09/25
Arun Ragavendhar Arunachalam Palaniyappan	Tech Lead, Developer	ARAP	06/09/25
Samsun Gulshan Sheik Dawood	Research Lead, Developer	SGSD	06/09/25
Vikas Kadruvan	Documentation Lead	VK	06/09/25

CLIENT SIGN OFF

Name	Position	Signature	Date
Organisation			

1. INTRODUCTION

The purpose of this document is to outline the concept design for the project “Benchmarking and Securing Language Models Against Adversarial Prompt Attacks.” It provides a clear understanding of the project's direction and anticipated results for all parties involved by outlining its history, scope, goals, methodology, and deliverables.

This document is mainly for the client, project supervisor, and team members, but it will also be useful for anyone with a role in shaping or reviewing the project. By bringing together the background, objectives, and planned approach, it provides a clear picture of how the team will move forward.

The report will act as a guide for managing tasks, resources, and responsibilities. Additionally, it will guarantee that moral principles are adhered to, risks are considered at an early stage, and the end results satisfy the client.

1.1. BACKGROUND

The Commonwealth Scientific and Industrial Research Organisation (CSIRO), Australia's national science agency, has initiated this project. The security of generative artificial intelligence is one of CSIRO's new focus areas. The organization is renowned for using innovation to solve challenging, real-world issues.

The use of large language models (LLMs) in academia, business, and research has raised serious concerns about their vulnerability to social engineering and adversarial prompts (OWASP, 2024). These include methods that have the potential to reveal private data or produce negative results, like role manipulation, hidden instructions, and prompt injection (Cohen & Heverin, 2025). CSIRO determined that it was necessary to investigate these risks in a methodical manner and consider possible defenses.

This project is being undertaken by our team as part of an industry-academic partnership. Our goals are to benchmark current models, create an attack repository while working under the direction of CSIRO and university supervisors. By this effort, we support CSIRO's overarching objective of guaranteeing the responsible and secure deployment of AI technologies.

1.2. KEY PROJECT PERSONNEL

The key personnel involve in this project are as follows:

Client

The project's client is CSIRO's Data61 division, which focuses on data science and digital technologies to solve complex national challenges. Dr. Chamikara Mahawaga Arachchige (Chamikara.Mahawagaarachchige@data61.csiro.au) has been our primary point of contact and has helped the team comprehend the project's expectations as well as the extent of adversarial prompt attacks.

Other Stake holders

In addition to Dr. Chamikara, we collaborate closely with Dr. Mohan Baruwal Chhetri (Mohan.Baruwalchhetri@data61.csiro.au) and Mr. Viet Vo (vvo@swin.edu.au) from Swinburne University, who supports the project from an academic perspective. Together, these stakeholders meet with our team weekly to review progress, provide feedback, and ensure that our work stays aligned with both CSIRO's objectives and the academic requirements of the unit.

Project Supervisor, Team Leader and Key Project Members

The project is supervised by Dr. Xinyi Cai, who provides academic guidance and ensures that the project outcomes align with unit expectations and ethical standards.

The team is led by Vaisakh Pariyacheri, who is responsible for coordinating meetings, managing timelines, and serving as the main contact point between the client and the group.

The other team members, Aarathi Narayanan Kadambatta, Aayudh Himanshubhai Vaghani, Arun Ragavendhar Arunachalam Palaniyappan, Samsun Gulshan Sheik Dawood, and Vikas Kadruvan share equal responsibilities in carrying out the technical work of the project. Each member contributes to the creation of the repository of adversarial prompt attacks, benchmarking of language models, and the development of defence strategies.

2. TERMS OF REFERENCE

Create an evidence-based, evolving systematization and prompts repository for social engineering attacks on LLMs. A complete system architecture diagram is shown in the figure below (figure 2.1). The project will determine which human social-engineering strategies are appropriate for prompt injection, how they materialize in prompts, and how consistently they function across models and circumstances.

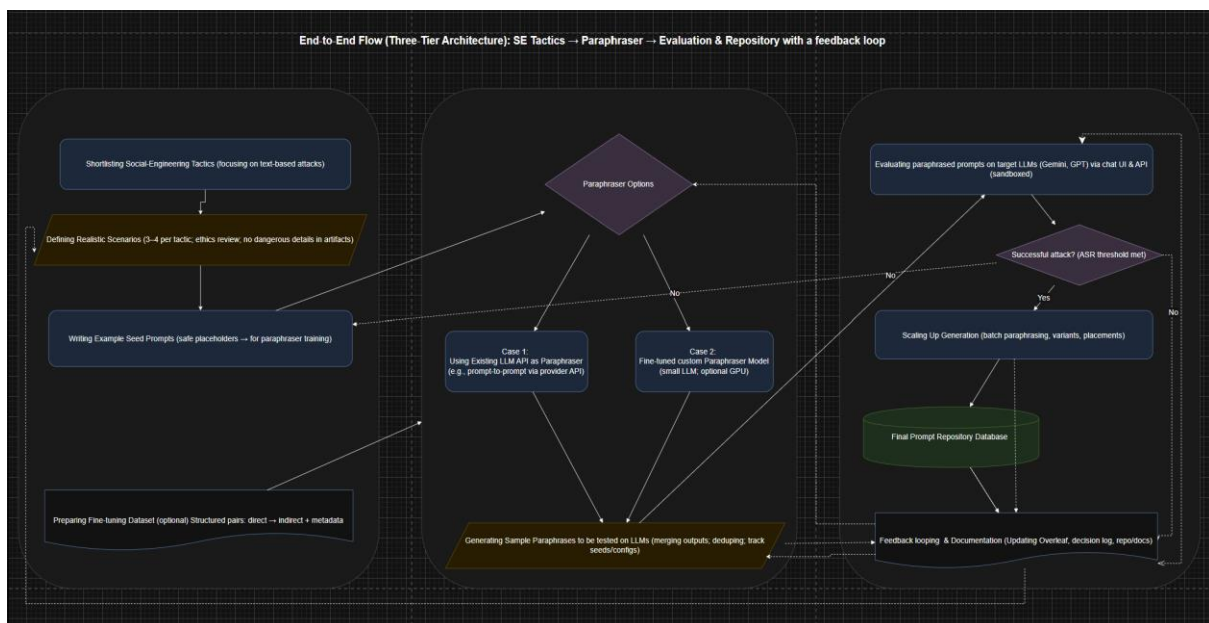


Figure 2.1. Proposed system design architecture

Intended users:

- Client/supervisors for research direction
- WP1 team (day-to-day contributors)
- Wider research group (follow-on WPs: WP2 and WP3) under controlled, ethical use.

2.1. OBJECTIVES

- Research & develop systematization: Define and document a system of classification that maps social-engineering strategies to prompt patterns and injection mechanisms, including formal definitions and examples for at least eight (8) different techniques.
- Produce training data for attack prompts: Curate ten or more documented attack patterns spanning at least four social engineering techniques
- Fine tuning a paraphraser LLM: Fine tune an LLM to paraphrase simple straight forward attack prompts into indirect ones using the developed training data and chosen social engineering techniques and attack patterns.
- Produce reproducible attack prompts: Generate a repository of prompt consisting malicious intent using the fine-tuned paraphraser LLM that uses the social engineering strategies and attack patterns chosen.
- Execute and report evaluation runs: Test the attack patterns across three or more distinct LLM families and report Attack Success Rate (ASR) along with other relevant metrics and transferability observations.
- Establish provability, ethics criteria and reporting: Define checklists and procedures for reproducibility, safety, and responsible use; ensure 100% of archived artifacts conform to these criteria. Produce a concise report summarizing methods, findings, limitations, and recommendations for subsequent work packages

2.2. SCOPE

The project will create and publish an evidence-based systematization that links social-engineering tactics to prompt patterns and injection mechanisms, with at least eight ways officially outlined. It will provide a training set of 10 or more documented attack patterns spanning at least four social-engineering strategies. The team will fine-tune a paraphraser LLM (if limited by the guardrails in the normal APIs to LLMs), which translates direct attack prompts into indirect, technique-aligned alternatives, and use it to create candidate prompts. These prompts will be assessed across three or more LLM families, with a minimum execution harness and objective assessors to provide an Attack Success Rate (ASR) as well as observations on transferability and concealment.

This project will not create or implement production-level defences, guardrails, or institutional policies. It will not engage in real-world phishing or ransomware activities, nor will it undertake non-sandboxed testing on human beings or systems. It does not allow the acquisition or processing of personally identifiable information or other sensitive data, web-scale scraping, or undisclosed system testing. Training new base LLMs from scratch is out of the scope; only allowed fine-tuning or API-based usage will be performed. Developing a full stack product beyond a basic assessment harness, such as full user interfaces or multi-tenant services, is likewise out of scope.

The work expects access, if available, to permitted LLM APIs or fine-tuning endpoints with adequate computing quota. All experiments will employ fixed seeds and setups when available, with complete transcript and parameter logging to allow for repetitions. The project will always adhere to academic ethics and the client's safety standards.

2.3. CRITICAL SUCCESS FACTORS

Taxonomy clarity is crucial. The project must create a precise classification system using at least eight distinct approaches. These strategies should be well-defined and non-overlapping so that the customer may comprehend and use the taxonomy without confusion.

Training data integrity is also critical. The project must present at least ten documented attack patterns encompassing more than four social-engineering tactics. Each pattern should be accompanied by complete metadata to allow for data reuse and independent auditing.

The fine-tuned model must constantly provide indirect, technique-aligned paraphrases of attack prompts. Its output should be of acceptable quality when tested on a small validation set, exhibiting both faithfulness to the original aim and adherence to the chosen technique.

Reproducibility and auditability are critical. All experiments must document inputs, configurations, random seeds, evaluation techniques, and transcripts (when accessible). A short redo of the trials should reliably reproduce pass/fail outcomes, confirming transparency and consistency. Evaluation breadth and insight must both be attained. Prompts will be tried on at least three separate LLM families, with the Attack Success Rate (ASR) and other findings carefully noted. This guarantees that the findings are not model-specific and offer a broader perspective.

Finally, safety and ethical compliance are required. The project must ensure that no personally identifiable information (PII) or harmful artifacts are present. Throughout the procedure, the team must closely follow the agreed-upon reproducibility and safety checklists (formulated and evolving throughout the project), ensuring responsible and ethical research practices.

2.4. ACCEPTANCE CRITERIA

The project is accepted when all (or many, after discussion on progress) of the following are true:

- The client approved the taxonomy, which includes at least eight approaches (definitions, examples, and mechanism mapping).
- Data and repository ready: Document more than 10 attack patterns across more than four approaches.
- The fine-tuned paraphraser produces technique-aligned indirect prompts on a small test set with acceptable quality (e.g., approximately 75% ASR on tests).
- Evaluation is complete: Prompts are conducted on more than three LLM families, with ASR and reported the observations, as well as a minimum framework and instructions.
- Compliance was met: the reproducibility and ethics requirements were completed; there was no PII or hazardous artifacts; and configurations/runs were logged.
- Report delivered: A summary of techniques, results, limitations, and future steps is supplied.

Acceptance testing methods:

- Taxonomy: Client receives the taxonomy document (more than techniques), checks a few sample classifications for clarity and completeness, and approves by email.
- Paraphraser/API paraphrasing: Client receives the demo (code/API), runs it, confirms output prompts and their ASRs and approves by email.
- Repository & Evaluation: Client receives the full repository (attack prompts & ASR report), can rerun a sample to confirm reproducibility on selected LLMs, and safety compliance, then approves by email.

3. ESTABLISHMENT

3.1. PROCESSES, PROCEDURES AND STANDARDS

Proposed methodology / guidelines (method + rhythm)

- An Agile sprint (2 weeks) is to be followed. Loop: discover, design, defend, document. The table below shows the planned rhythm for a 2-week sprint. It is a guideline: the team may follow it strictly or adjust activities and outputs as the project progresses.

Day	What happens	Concrete outputs
Mon (AM)	Sprint planning (45–60 min)	Sprint brief, selected tactics/scenarios, owners
Mon (PM)	discover	Risk updates, interview/survey plan (if any)
Tue	design	Prompt templates, 10 prompts per tactic started.
Wed	design → defend hand-off	Configs ready; defence text final; Peer review
Thu	defend (baselines)	asr.csv, qa_scores.csv, sum_overlap.csv (baseline)
Fri	defend (defences)	Same files for Boundary / Reminder / Both
Mon (wk2)	defend (repeats, seeds, edge cases)	Completed runs; Testing the .md files
Tue	document	Summary tables + two slides
Wed	internal review (30 min)	Sign-off on findings; list next-sprint items
Thurs	retro (20 min)	What to keep/change next sprint
Fri	client/supervisor demo (15–30 min)	Decisions captured in decision log

Table 3.1: Sprint execution template

- Focus of WP1:
 - The core idea is to do a suitability analysis across many social-engineering items.
 - Keeping only tactics that can be used in text prompts, then go deep on those (scenarios, prompts, tests, measures).
- Regular Schedule: weekly stand-up; end-of-sprint demo and decision log; risk review during retro.

- Change control: all work via issues + PRs; Definition of Ready (clear goal, test plan) and Definition of Done (tests pass, docs updated).

Requirement elicitation

A. Participant Information and Consent

- Provide all participants with a plain-English information sheet and a consent form.
- The sheet will clearly explain the purpose of the study, that participation is voluntary, and that participants can withdraw at any time without any consequence.

B. Minimising Bias in Questions

- Design interview/survey questions carefully to avoid leading or suggestive wording.
- Only neutral, open-ended questions will be used.
- Demographic questions will be optional and include a “prefer not to say” option.
- A pilot run will be conducted with one team member first, to check that the form is clear and unbiased.

C. Data Collection and Storage

- To be done: Collect only the information directly relevant to the project.
- Notes will be de-identified so no personal names or identifiers remain.
- Data will be stored securely in private, access-controlled university folders.
- All data will be deleted on the specific date stated to participants in the consent form.

Suitability analysis (what is included/excluded)

Tactics that can be written as persuasive text inside or around a prompt are kept. Outcomes, channels, and physical-world items will be dropped.

A. Included tactics (Tactics to be studied):

- Baiting, Pretexting, Impersonation (e.g., Help Desk/Admin), Quid-pro-quo/Reciprocity, Authority, Social proof, Urgency/Scarcity, Story/Empathy framing, Reverse social engineering.

B. Not used as tactics (document only):

- Outcomes: Phishing, Ransomware.
- Channels: Robocalls, SMS phishing, Pop-ups, Phone SE.
- Physical: Tailgating, Dumpster diving, Shoulder surfing.

C. Exact steps to be followed

1. Label each tactic: tactic / outcome / channel / physical.
2. Keep only tactic. For each kept item, write a one-line purpose and where it would sit (system message, user message, or retrieved text).
3. For each kept item, design 4 scenarios (short, realistic).
4. For each scenario, generate at least 3 prompts to train an LLM, which can be used as a paraphraser. This approach will be done if current LLM guardrails doesn't allow paraphrasing harmful prompts via APIs. Estimated training data size: 9 tactics × 4 scenarios × 3 prompts = 108 prompts (manageable, deep enough).

5. The fine-tuned paraphraser (or the LLM API) can be then used to generate more prompts as mentioned below.

Prompt generation (clear, controlled, reproducible)

Prompt creation will be done in a clear, controlled, and reproducible manner, either by fine-tuning a small LLM to act as a paraphraser or by using existing LLM APIs, depending on available compute resources. Arun will lead this process, beginning with 108 manually constructed questions as samples and growing the dataset by producing new paraphrased alternatives. The vast number of prompts generated by this technique will serve as the input set for evaluating social-engineering attack strategies across three separate LLM families, assuring broad evaluation coverage and consistent results.

Evaluation plan (what is tested and how scoring is to be done)

- Malicious content placements: inject at start, middle, end of the context.
- Surfaces: prompts injected with malicious intent, either paraphrased by a fine-tuned paraphraser or paraphrased via an LLM API.
- Metrics:
 - Attack Success Rate (ASR) = number of harmful/forbidden behaviours that appear ÷ number tested.

Cost-aware defence plan (simple first, heavy later if needed)

- Default = black-box prompt patterns (boundary awareness + explicit reminders).
- If a few tactics still bypass these, add a light guard to screen those cases.
- Only if results are still weak on a must-fix scenario, consider a small fine-tune on our synthetic set (time-boxed, GPU optional).

Versioning, QA, and coding rules (exact and minimal)**A. Proposed repository layout**

- /docs – Holds project documentation, including plans, meeting notes, and the decision log.
- /prompts – Stores all prompt files, organised by tactic and scenario.
- /data – Contains clean input data, such as corpora or retrieved text snippets, with no personal information (PII).
- /experiments – Keeps configuration files for experiments, including task settings, placements, seeds, and defence options.
- /eval – Stores evaluation results such as scores, plots, and run summaries.
- /scripts – Holds code scripts for generating prompts, running experiments, and scoring results.

B. Versioning, QA, and Coding Rules

- Set clear rules for how code is stored, reviewed, tested, and reproduced.
- Make sure the repo is secure, consistent, and easy to repeat runs from.
- Git rules:
 - Keep the main branch protected.
 - Use feature branches and fix branches (e.g., feat/, fix/).
 - Every pull request (PR) will require at least one peer review before merging.
- Code practices:
 - Use Python with proper testing.

- Write clear comments for readability.
 - Ensure no secrets are stored in code or prompts.
- Reproducibility:
 - Every run will include a config file and a seed number.
 - Results will be stored systematically under /eval/<date>/<run_id>.

IP, copyright, attribution (simple rules)

- Keep an Attribution Log (source, licence, date) for any figure, dataset, or taxonomy we use.
- Prefer open licences.
- In the report, using Harvard style referencing style, add citations to statements referred from another research.
- If we adapt a figure, label it “Adapted from ...”.

3.2. PROJECT ENVIRONMENT

- Workplaces: hybrid (campus + home).
- Storage: private OneDrive/SharePoint.
- Compute: laptops (≥ 16 GB RAM) + Docker.
- GPU: only if the optional fine-tune is to be done; otherwise not needed.
- Tools: GitHub (issues, PRs, Actions), Teams/Discord (chat), Canvas(submission), Trello (project management)
- Access: role-based; Safety Lead must approve any change to guard prompts or safety settings; secrets only in CI variables (not in code).

3.3. PROJECT TEAM SKILL DEVELOPMENT REQUIREMENTS

The project team will need ethical skills like as permission, plain-English communication, de-identification, secure storage, and understanding when to escalate for review. They also require collaboration abilities for efficient teamwork. In attack prompt design, the team must learn to generate safe but realistic prompts using social-engineering techniques, experimenting with location to maximize success.

Evaluation skills are required for assessing outputs and compute the Attack Success Rate (ASR). Along with this, the team must manage the repository and CI/CD operations, such as branch flow, testing, and GitHub.

Strong attribution and reporting methods are also required, including accurate sources, sprint notes, clear tables/plots, and brief summaries. Finally, the team should learn how to interpret outcomes in straightforward language for supervisors, as well as exercise collaborative practices including code reviews, conflict resolution, and shared decision-making.

4. DELIVERABLES, ACTIVITIES AND CAPITAL RESOURCES

4.1. DELIVERABLES

The project will produce the following deliverables:

- Classification and Taxonomy Documentation: A formal classification mapping at least eight social-engineering strategies to prompt patterns and injection mechanisms, with definitions

and examples. This taxonomy will provide a systematised view of how human-oriented manipulation techniques translate into LLM-oriented attacks.

- **Training Dataset of Documented Attack Patterns:** A curated dataset of at least 10 representative attack patterns spanning ≥ 4 social engineering techniques, with full metadata for reproducibility.
- **Fine-tuned Paraphraser LLM / Prompt Generation Pipeline:** A paraphraser model (or reproducible API pipeline) that converts direct harmful prompts into indirect, technique-aligned adversarial prompts, validated on a small test set.
- **Repository of Social Engineering Prompt Injection Attacks:** An evolving repository of adversarial prompts generated using the fine-tuned paraphraser LLM, refined for stealth and effectiveness, and archived with metadata.
- **Evaluation Report:** Documented attack success rates, transferability observations, and metrics across three or more LLM families.
- **Provability & Ethics Checklist:** A reproducibility and safety framework including logged seeds/configurations, ethical safeguards, and compliance documentation.
- **Presentation Materials:** Slides/poster summarising methodology, results, limitations, and recommendations for future work packages.

4.2. ACTIVITIES

The project will follow a staged approach aligned with the research's sprint lifecycle:

Tasks for each activity are given below.

Activity 1: Literature Review and Scoping

- Reviewing existing academic papers on social engineering, persuasion, and prompt injection to synthesise new information to be used in the project.
- Extracting candidate social engineering techniques.
- Conducting a suitability analysis to identify which techniques can plausibly apply to LLM attacks.

Activity 2: Classification and Taxonomy Development

- Defining categories of social engineering techniques applicable to prompt injection.
- Mapping techniques to corresponding prompt patterns and attack mechanisms.
- Drafting taxonomy diagrams and descriptive documentation.

Activity 3: Training Data Generation

- Designing repository schema (fields for technique, prompt, injection type, metadata).
- Populating the dataset with at least 10 representative attack patterns spanning ≥ 4 techniques.
- Ensuring reproducibility by documenting prompts, context, and model configurations.

Activity 4: Paraphraser Model Development

- Using the curated training data to fine-tune an LLM (or configure an API-based paraphrasing pipeline).
- Generating indirect adversarial prompts aligned with chosen techniques.
- Validating outputs on a small test set, measuring alignment and ASR quality.

Activity 5: Prompt Generation and Refinement

- Using the fine-tuned paraphraser to generate indirect prompts.
- Iteratively refining prompts for stealth and diversity.
- Storing validated prompts in the repository with metadata.

Activity 6: Provability and Ethics Compliance

- Defining reproducibility checklist (seeds, configs, transcripts, metadata).
- Ensuring safety guardrails (no PII, no hazardous artifacts).
- Validating that repository entries meet compliance criteria before final archiving.

Activity 7: Evaluation and Analysis

- Testing generated prompts across at least three LLM families.
- Measuring attack success rate (ASR), transferability, and stealth characteristics.
- Documenting comparative findings and key observations.

Activity 8: Reporting, Presentation and Interpretation

- Compiling a structured report including methods, results, limitations, and ethical safeguards.
- Preparing poster/slide deck for presentation to supervisors and stakeholders and interpreting the project as per client's understanding.

4.3. RESOURCES

To complete the project, the following resources will be required:

A. Software/Platforms

- Access to LLM APIs (e.g., OpenAI GPT, Anthropic Claude, Google Gemini, or open-source models such as LLaMA).
- Code repository and collaboration tools (e.g., GitHub, Overleaf, MS Teams).
- Data management and documentation tools (e.g., Excel, Notion, or JSON/YAML repository format).

B. Hardware/Infrastructure

- Standard computing resources (university-provided machines or personal laptops).
- Cloud credits if fine-tuning or large-scale testing is required (AWS/Oracle Cloud).

C. Knowledge Resources

- Academic literature on social engineering, persuasion, and AI safety.
- Supervisor guidance and feedback.

D. Facilities

- University-provided workspaces, meeting rooms, and online collaboration platforms (Zoom, Canvas, Teams).

5. ORGANISATION AND STRUCTURE**5.1. PROJECT TEAM (GROUP XC2/WP1 – 6 TEAM MEMBERS)**

- Carry out the project work: research, design, development, testing and documentation.
- Roles (can be rotated/shared):

- Project Manager / Coordinator – schedules meetings and ensures deadlines.
- Research Leads – conduct literature review and suitability analysis.
- Technical Leads – implement repository and testing framework.
- Documentation Leads – maintain repository documents and reports.

5.2. INDUSTRY CLIENTS (3 REPRESENTATIVES FROM CSIRO DATA61)

- Act as external stakeholders.
- Provide domain expertise and validate alignment with industry needs.
- Review intermediate deliverables and provide feedback.
- Sign-off on acceptance testing.

5.3. SWINBURNE ACADEMIC FACILITATOR

- Ensures academic requirements are met.
- Acts as bridge between project team and clients.
- Provides feedback on assignment submissions.

5.4. TESTERS / EVALUATION USERS

- Team members (and possibly clients/facilitator in observation role).
- Run generated prompts against LLMs and document attack outcomes.

The project adopts a matrix organisational structure, where activities and deliverables are shared across the project team, industry clients, and academic facilitator. Table 1 presents the RACI matrix followed in this project, which is a framework with a matrix structure that defines roles and responsibilities across key project activities, ensuring clarity for all participants and stakeholders working together [3].

Each deliverable is mapped to activities and assigned clear roles:

- Responsible (R) for execution.
- Accountable (A) for sign-off.
- Consulted (C) for providing feedback.
- Informed (I) for being updated on progress.

This structure ensures that the student team maintains ownership of technical work, while the clients provide domain validation and the facilitator ensures academic compliance. The RACI format clearly maps every activity to be done to a specific deliverable that is going to be provided and who exactly provides it.

Activities	Lit. Review & Scoping	Taxonomy Dev.	Training Data Gen.	Paraphraser Dev.	Prompt Refinement	Ethics Compliance	Evaluation & Analysis	Reporting & Presentation
Deliverables								
Classification & Taxonomy Documentation	R (Team) / C (Clients)	R (Team) / A (Facilitator)	–	–	–	–	–	I (Clients)

Training Dataset of Attack Patterns	–	–	R (Team) / C (Clients)	–	–	A (Facilitator)	–	I (Clients)
Fine-tuned Paraphraser LLM / Pipeline	–	–	–	R (Team) / C (Clients)	–	–	A (Facilitator)	–
Repository of SE Prompt Attacks	–	–	R (Team)	R (Team)	R (Team) / C (Clients)	A (Facilitator)	–	–
Evaluation Report	–	–	–	–	–	–	R (Team) / A (Facilitator)	C (Clients)
Provability & Ethics Checklist	–	–	–	–	–	R (Team) / A (Facilitator) / C (Clients)	–	–
Presentation Materials	–	–	–	–	–	–	–	R (Team) / C (Clients) / A (Facilitator)

Table 5.4: RACI matrix for Activities and Deliverables

6. RISKS

Risks associated with this project are mentioned in the table below.

Rank	Name / Description	Occurrence Probability (H/M/L)	Severity (H/M/L)	Mitigation Strategy Number	Contingency
1	Data Availability Issues: Limited access to real-world adversarial prompts or restricted datasets may slow progress.	M	H	1	Use synthetic data generation and publicly available repositories as substitutes.
2	Technical Challenges with LLMs: Difficulty in running large models due to compute or compatibility issues.	M	H	2	Shift to smaller open-source models (e.g., LLaMA, Falcon) and cloud resources.
3	Ethical and Security Concerns: Risk of producing harm	L	H	3	Follow CSIRO and university ethical guidelines; restrict repository access.

	ful prompts or unintended misuse during testing.				
4	Time Management Project: tasks (repository, benchmarking, defences) taking longer than expected.	M	M	4	Re-prioritise deliverables, divide tasks evenly, and use weekly progress reviews.
5	Stakeholder Availability Delays if clients or supervisors are unavailable for regular meetings/feedback.	L	L	5	Use email/Discord updates and adjust timelines to align with their availability.
6	Tool or Platform Failures: Issues with versioning systems, servers, or testing platforms.	L	M	6	Maintain backups, adopt alternative tools, and log experiments for reproducibility.
7	Team Member Unavailability: A member missing weekly meetings due to illness or personal commitments.	L	M	7	Record meeting notes and share updates through group channels to keep everyone aligned.
8	Uneven Contribution: Some members may contribute less, affecting workload balance.	L	H	8	Allocate clear tasks with deadlines and review progress weekly. If needed, redistribute tasks fairly.
9	Requirement Changes: Client adjusts scope or deliverables midway during project.	L	H	9	Reconfirm scope with client and supervisor; update plan accordingly.
10	Misalignment Between Client and Team: understanding of project outcomes.	L	H	10	Weekly confirmations and meeting notes to align expectations.
11	Skill Gaps Within Team: Limited expertise in AI security or benchmarking.	L	M	11	Schedule training, share resources, simplify scope if needed.

12	Dependency on External Tools: API limits or model access changes.	L	M	12	Maintain backup tools/models; test alternatives early.
13	Communication Breakdowns in Team: Poor updates or unclear task ownership.	L	M	13	Clear channels (Trello/Teams) and weekly progress checks.

Table 6.1 - Risks

7. SCHEDULE

7.1. PROJECT TIMELINE

The project timeline is represented using a Gantt chart as shown in Figure 7.2, showing each activity as a horizontal bar to show the sequence and duration. The activities are shown in blue while the external dependencies are represented in red, that must occur before downstream tasks can proceed smoothly.

Full Gantt chart link:

<https://app.ganttpro.com/shared/token/8da973ad75d2b8f9f8ea1c21768f3114c00747fa73adb62e0dd92e4bba64ef36/1915292>

Task name	Start date	End date	Assignees
	11/08/2025	30/10/2025	
1 ☐ Activities	11/08/2025	30/10/2025	
1.1 Literature Review and Scoping	11/08/2025	25/08/2025	Vaisakh, Arun
1.2 Classification and Taxonomy Development	25/08/2025	08/09/2025	Aarathi, Samsun
1.3 Training Data Generation	08/09/2025	15/09/2025	Arun, Aayudh
1.4 Paraphraser Model Development	15/09/2025	29/09/2025	Samsun, Vikas
1.5 Prompt Generation and Refinement	29/09/2025	07/10/2025	Vaisakh, Aarathi
1.6 Provability and Ethics Compliance	07/10/2025	15/10/2025	Vikas, Arun
1.7 Evaluation and Analysis	15/10/2025	23/10/2025	Aayudh, Vaisakh
1.8 Reporting and Presentation	23/10/2025	30/10/2025	All
2 ☐ External Dependencies	01/09/2025	09/10/2025	
2.1 Client Feedback	01/09/2025	05/09/2025	Chamikara
2.2 Access to resources	08/09/2025	12/09/2025	Swinburne
2.3 Facilitator Input	03/10/2025	09/10/2025	Xinyi Cai

Table 7.1: Project Delivery Timeline

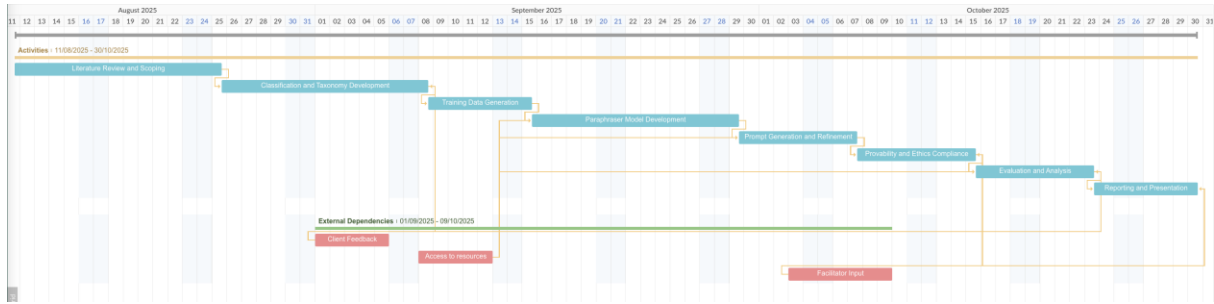


Figure 7.2: Gantt Chart

7.2. EXTERNAL DEPENDENCIES

The schedule relies on three critical external inputs, each shown in the Gantt chart in Figure 7.2:

- Clients' feedback: critical for suitability analysis, repository validation.
- Facilitator's academic input: needed for approval before submission deadlines.
- Access to LLM APIs/tools: if restricted, development timeline may be delayed.

7.3. ASSUMPTIONS

- All team members and clients remain available throughout the semester.
- Clients respond to feedback requests within a few days.
- LLM APIs and other resources remain accessible during development phase.
- Facilitator is available for milestone reviews and assignment guidance.

8. BUDGET

8.1 PERSONNEL COST

Name	Role	Rate per Hour (AUD)*
Vaisakh Pariyacheri	Team Lead	45
Aarathi Narayanan Kadambatta	Developer	45
Aayudh Himanshubhai Vaghani	Developer	45
Arun Ragavendhar Arunachalam Palaniyappan	Tech Lead, Developer	45
Samsun Gulshan Sheik Dawood	Research Lead, Developer	45
Vikas Kadruvan	Documentation Lead	45

Table 8.1: Personnel Cost

*Rate per hour: Approximate research/software assistant rate for university projects.

Disclaimer on Budget Responsibility

The budget outlined above is for internal planning and academic reporting purposes only. All listed costs, including but not limited to personnel time, software, API usage, and computing resources, will be absorbed by the research team and/or the university. The client will not be invoiced or held financially responsible for any portion of this budget, either directly or indirectly.

8.2. TIME ESTIMATED TO COMPLETE EACH TASK

The breakdown below aligns each activity with its project schedule (Section 7.1) and assumptions (10 hours/week/per member, 6 members × ~10 weeks = ~600 person-hours max for project work):

Activity	Task	Estimated hours needed (hrs)	Total per activity (hrs)
1.1	Literature Review and Scoping	60	
1.2	Classification & Taxonomy Dev.	70	
1.3	Training Data Generation	70	
1.4	Paraphraser Model Development	80	
1.5	Prompt Generation & Refinement	60	
1.6	Provability & Ethics Compliance	40	
1.7	Evaluation and Analysis	50	
1.8	Reporting and Presentation	30	460
2.1	Client Feedback (External Dependency)	10	
2.2	Access to Resources (Ext. Dep.)	8	
2.3	Facilitator Input (Ext. Dep.)	8	26
	Total		486

Table 8.2: Time estimates for the tasks.

Estimates reflect group effort, concurrency, and allow for some slack as illustrated in the schedule. Adjusted to ensure feasibility within the timeline and maximum total team hours available.

Budget calculation example:

Total hours (Activities + Dependencies) = 486 hrs.

Hourly rate = \$45.

Total Personnel Cost: 486 hrs × \$45/hr = \$21,870

8.3. BUDGET GUIDANCE NOTES

- They should equally contribute as each member of the team, according to the project rules and agile process described in the project plan.
- The hours per activity are identical to the workflow and project schedule in Section 7.1, including both slack time and the external dependencies.
- Personnel rates are based on typical university research or technical project rates of student-led software projects in Australia.
- The cumulative hours are also meant not to be more than project limit (10 hrs/week/member × 6 employees × -approximately-8 weeks of main working period plus some buffer).

This budget plan can be varied when the institution has established other hourly rates. Student project work of this level of difficulty in Australia often costs around \$35 to \$50/hr based on technical/research needs and university policies.

9. REFERENCES

1. Cohen, E., & Heverin, T. (2025). *Evaluating the effectiveness of psychological prompt injection attacks on large language models for social engineering artifact generation*. In *Proceedings of the 24th European Conference on Cyber Warfare and Security (ECCWS 2025), Work-in-Progress Papers* (pp. 879–883). Academic Conferences International. <https://doi.org/10.34190/eccws.24.1.3515>
2. Open Web Application Security Project (OWASP). (2024). *OWASP top 10 for LLM applications 2025*. GenAI.OWASP.org. <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>
3. Matthews, Bradon. "What Is a RACI Matrix? Definition, Examples, Uses." *Project-Management.com*, 19 Apr. 2024. <https://project-management.com/understanding-responsibility-assignment-matrix-raci-matrix/#what-is-a-raci-matrix>